SolarBI: Power BI y Grafana, gobernanza de datos, automatización ETL e IoT

Institución Universitaria Pascual Bravo

Inteligencia de Negocios · Grupo 50

Docente: Ramiro Grisales Montoya

Integrantes: Laura Mejia Delgado y Yuritza Mendoza

Fecha: 9/10/2026

Repositorio: https://github.com/lauramejia583-design/solarbi-Mejia-Mendoza.git

Parte A · Consulta

Bloque 1 · Power BI y Grafana

D1. ¿Qué es Grafana?

Grafana es una plataforma de código abierto que permite la visualización, el monitoreo y el análisis de datos a través de gráficos, tablas y paneles interactivos conocidos como dashboards. Su uso principal es la supervisión del rendimiento de servidores, redes, aplicaciones y bases de datos, lo que ayuda a detectar errores, seguir métricas y configurar alertas en caso de que surjan problemas. Además, tiene la capacidad de conectarse a diversas fuentes de datos, como Prometheus, MySQL y SQL Server, para presentar la información de manera clara y ordenada.

D2. Power BI vs. Grafana

Criterio	Power BI	Grafana
Propósito principal	Análisis de negocio y reportes	Monitoreo y observabilidad
Usuario típico	Analista, directivo	Operador, ingeniero de planta
Conexión a los datos	Conectores; copia (Import) o consulta directa	Consulta directa a cada data source
Modelo semántico y lenguaje	Modelo en memoria con relaciones; DAX	Sin modelo propio; SQL (o el lenguaje del origen)
Actualización y tiempo real	Refresco programado; tiempo real limitado	Auto-refresh de segundos; ideal para tiempo real
Alertas	Alertas en visuales del servicio, limitadas	Reglas de alerta con canales de notificación
Licenciamiento y costo	Desktop gratis; servicio con licencias Pro/PPU/Fabric	OSS gratis; Cloud con plan gratuito y de pago
Control de acceso	Roles del servicio y seguridad a nivel de fila (RLS)	Usuarios, equipos, roles y permisos por carpeta
Versionado	.pbix binario o .pbip	Dashboards en JSON

D3. Tipos de dashboard y KPIs

KPI	Tipo	Herramienta	Por qué
Energía (kWh)	Analítico	Power BI	Se compara por día y mes; la potencia de 5 min va en Grafana
Yield y Performance Ratio	Analítico/estratégico	Power BI	Requieren cálculo con referencia fija (kWp) e histórico

Ahorro (COP)	Estratégico	Power BI	Lo mira el directivo; usa un parámetro de tarifa
Disponibilidad y alarmas	Operativo	Grafana	Exige datos frescos y alertas
% de datos válidos	Operativo	Grafana	Se vigila con umbrales de color

D4. Import, DirectQuery y actualización

En Import Power BI guarda una copia de los datos en el modelo: es rápido y permite todo DAX, pero solo cambia al refrescar. En DirectQuery cada visual consulta la fuente en vivo: datos al día, pero más lento y con funciones limitadas. La actualización programada vuelve a importar a horas fijas. En SolarBI usaríamos Import: fact_energia_dia es pequeña y se actualiza una vez al día tras el ETL de medianoche. El auto-refresh de Grafana, en cambio, repite las consultas en vivo cada pocos segundos, sin copia local

D5. Variables y segmentadores

Una variable de Grafana (templating) es un valor elegido en una lista que se inserta en el texto de la consulta; Grafana lo reemplaza y el filtrado ocurre en la base de datos. Un segmentador de Power BI filtra el modelo ya cargado mediante el contexto de filtro de DAX. Variable propuesta, dispositivo, con la consulta de la lista y su uso en el panel:

```
-- Variable $dispositivo
SELECT DISTINCT dispositivo_id FROM silver.lectura_5min ORDER BY 1;
-- Panel (la consulta cambia según el dispositivo elegido)
SELECT ts AS time, p_ac_kw FROM silver.lectura_5min
WHERE $__timeFilter(ts) AND dispositivo_id = $dispositivo ORDER BY ts;
```

Bloque 2 · Gobernanza de datos

G1. Data governance

Según el marco DAMA-DMBOK, la gobernanza de datos es el ejercicio de autoridad y control (políticas, roles y decisiones) sobre la gestión de los activos de datos [3]. La *gestión de datos* (data management) es el conjunto de actividades que ejecutan esas reglas de almacenar, integrar, modelar, respaldar. La *calidad de datos* es una de esas áreas y mide si los datos sirven para su uso (validez, completitud, unicidad). En SolarBI, la gobernanza decide que una potencia negativa es inválida y quién lo aprueba, la gestión es el ETL que la filtra, la calidad es el porcentaje de datos válidos.

G2. Roles de gobernanza y del equipo

En la práctica, la gobernanza reparte responsabilidades clave: el Data Owner decide qué se hace con los datos, el Data Steward vigila su calidad funcional y el Data Custodian administra la infraestructura técnica. En SolarBI, Laura y Yuritza asumimos estos roles de forma compartida. Como Data Owners fijamos las metas estratégicas del proyecto. Desde el

rol de Data Stewards (analistas y modeladoras BI) definimos la regla de rechazar lecturas de potencia negativas y creamos las fórmulas DAX para los KPIs. Finalmente, al actuar como Data Custodians (ingenieras de datos), garantizamos la estabilidad de PostgreSQL y la ejecución del script Python.

G3. Contrato de datos (Data Contract)

Un contrato de datos es el acuerdo técnico que asegura que la fuente entregue la información como se espera. En SolarBI, fija que `telemetria.csv` envíe lecturas cada 5 minutos con columnas exactas (`ts`, `dispositivo_id`, `p_ac_kw`, etc.), tipos de datos correctos, unidades en kW y un mínimo de 90% de registros válidos. Si el origen cambia de repente el nombre de una columna o altera las unidades sin avisar, el flujo en Python fallará al procesar. Como resultado, la base de datos no se actualizará y los tableros en Power BI y Grafana quedarán rotos o vacíos.

G4. Gobernanza de indicadores y catálogo de KPIs

Contar con una única fuente de verdad significa que todos en la organización consultan los mismos datos centrales para evitar cifras contradictorias. El catálogo de KPIs formaliza esta lógica documentando cada indicador:

KPI	Fórmula	Unidad	Fuente	Responsable	Frecuencia
Energía Diaria	$\sum(p_ac_kw \times \frac{5}{60})$	kWh	dwh.fact_energia_dia	Data Engineer	Diaria
% Datos Válidos	$\left(\frac{FilasVálidas}{FilasTotales}\right) \times 100$	$\%$	dwh.fact_energia_dia	BI Analyst	Diaria / 5 min

Si calculamos la energía en SQL para Grafana agrupando por sumas simples y en DAX para Power BI aplicando filtros distintos de horario, obtendremos métricas desalineadas que perderán la confianza de los usuarios.

G5. Ley 1581 de 2012 y ética de datos

Las métricas puramente técnicas del inversor (potencia en kW o irradiancia en W/m^2) no son datos personales. Sin embargo, si el tablero llega a registrar el nombre o la identificación del operador que atendió una falla específica, esa información pasa a ser un dato personal bajo la Ley 1581 de 2012 de Colombia, ya que permite evaluar el desempeño laboral de un individuo. Nuestra postura en SolarBI es proteger la privacidad, si llegamos a monitorear la gestión de alarmas, utilizaremos códigos anónimos y restringimos el acceso mediante roles para garantizar un uso ético y legal.

Bloque 3 · Automatización de procesos ETL

E1. ETL vs. ELT y Power Query

La diferencia radica en dónde ocurre la limpieza. En ETL los datos se transforman en memoria antes de guardarlos, en ELT se suben crudos a la base de datos y se procesan dentro usando SQL.
En SolarBI usamos ETL: nuestro script en Python limpia el CSV en memoria y entrega solo los datos validados a PostgreSQL.
Respecto a Power Query, sí es una herramienta ETL genuina extrae datos de diversas fuentes, les aplica transformaciones complejas con su motor M en memoria y finalmente los carga en el modelo semántico del reporte.

E2. Carga completa vs. incremental e idempotencia

La carga completa borra y vuelve a insertar toda la base de datos en cada ejecución, mientras que la carga incremental solo añade los datos nuevos recibidos desde la última lectura. Que una carga sea idempotente significa que podemos correr el proceso varias veces sin alterar el resultado ni duplicar filas. En SolarBI lo garantizamos combinando la clave primaria (`fecha`, `dispositivo`) en `dwh.fact_energia_dia` con la instrucción SQL `INSERT ... ON CONFLICT DO UPDATE` (UPSERT). Así, al volver a ejecutar `run_etl.py`, el sistema actualiza la fila existente en lugar de crear una repetida.

E3. Cuadro comparativo de automatización

Criterio	Cron / Programador de Tareas	Apache Airflow	Actualización Programada Power BI
¿Qué es?	Herramienta nativa del sistema operativo.	Plataforma para orquestar flujos mediante código Python.	Servicio en la nube para actualizar reportes analíticos.
¿Qué automatiza?	Scripts de consola o tareas por hora fija.	Tuberías complejas con dependencias y reintentos.	Importación periódica del modelo semántico.
¿Cuándo conviene?	Tareas simples locales como correr `run_etl.py` a medianoche.	Entornos corporativos con múltiples sistemas interconectados.	Publicación de tableros que consumen datos ya procesados.

E4.Actualización incremental en Power BI

Esta función trabaja definiendo dos parámetros especiales de fecha: `RangeStart` y `RangeEnd`. Power BI utiliza estos rangos para consultar dinámicamente solo la porción de datos más reciente de la fuente, manteniendo intacto el histórico acumulado. Vale totalmente la pena implementar este esquema en proyectos solares cuando acumulamos años de lecturas cada 5 minutos (millones de registros), ya que evita la sobrecarga de red,

reduce drásticamente el tiempo de refresco y minimiza el consumo de memoria en el servidor en comparación con recargar la base completa diariamente.

Bloque 4 · Internet de las Cosas (IoT)

I1. Arquitectura IoT por capas

La arquitectura IoT se compone de cinco capas interconectadas:
1. Dispositivo → 2. Gateway/Edge → 3. Red/Broker → 4. Almacenamiento → 5. Aplicación.
En nuestro proyecto, el viaje del dato funciona así: la lectura nace en los sensores del inversor (Dispositivo), el Gateway local la colecta y genera el archivo `telemetria.csv` (**Bronze**), el broker la transmite por red hacia nuestro script de limpieza en Python, que la almacena filtrada en `silver.lectura_5min` (**Silver**), luego se consolida el acumulado en `dwh.fact_energia_dia` (**Gold**) y finalmente se despliega en los paneles de Grafana y Power BI en la capa de Aplicación.

I2. Protocolo MQTT

MQTT es un protocolo de mensajería liviano diseñado para redes de ancho de banda limitado. Opera bajo el modelo publicador/suscriptor, los sensores publican mensajes en un broker central organizados bajo rutas jerárquicas llamadas topics, y los sistemas interesados se suscriben para recibirlos. Maneja tres niveles de calidad de servicio que son QoS 0 (se envía sin confirmar), QoS 1 (asegura entrega al menos una vez) y QoS 2 (garantiza entrega exacta sin duplicados).
Para la planta solar proponemos la siguiente jerarquía:
`pascualbravo/solar/campus_franja/inversor_01/p_ac_kw`

I3. Comparación de protocolos IoT

Criterio	Modbus (RTU/TCP)	MQTT	OPC UA
Modelo	**Maestro / Esclavo**	**Publicador / Suscriptor**	**Cliente / Servidor interconectado**
Uso típico	Comunicación cableada entre equipos de planta	Transmisión remota de telemetría hacia la nube	Integración industrial compleja y SCADA
Formato	Registros numéricos en binario/hexadecimal	Cargas útiles flexibles en JSON o texto	Objetos con información contextual detallada
Seguridad	Inexistente en RTU; básica en TCP	Cifrado TLS y autenticación por contraseña	Certificados digitales y cifrado robusto

En una instalación solar real conviven dos niveles: Modbus RTU/TCP para comunicar los inversores con el Gateway local, y MQTT para enviar los datos del Gateway a la nube.

I4. Batch vs. Streaming y agregación de datos

Las lecturas recibidas cada 5 minutos representan un flujo continuo (streaming), mientras que la tabla diaria `fact_energia_dia` procesa los datos por lotes (batch). Al sumar la energía por día ganamos rapidez en las consultas, menor tiempo de carga en reportes gerenciales y un almacenamiento eficiente, pero perdemos el detalle fino de caídas momentáneas por nubosidad. Grafana se adapta mejor a la frecuencia de 5 minutos porque está optimizado para monitorear tendencias inmediatas, mientras que Power BI destaca analizando las tendencias agregadas por día o mes.

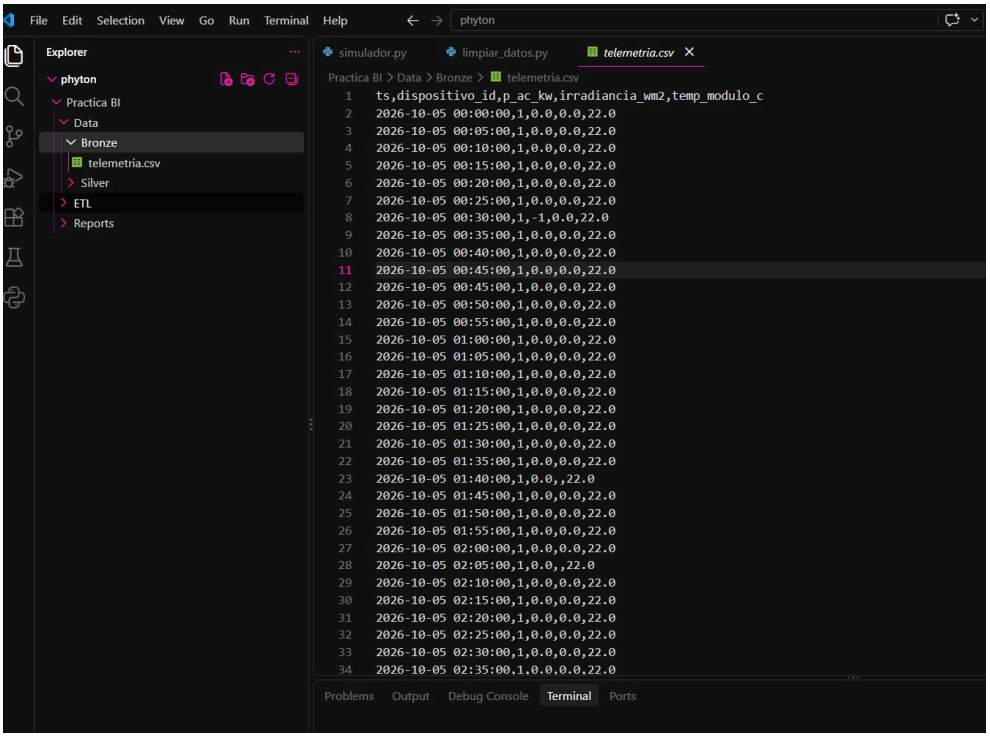
Bloque 5 · Control de versiones

V1. Git, GitHub y versionado de tableros

Git es la herramienta local que rastrea los cambios en el código, mientras que GitHub es el servicio en la nube donde alojamos y compartimos el proyecto. En este flujo, un repository es la carpeta del proyecto, un commit es una foto de los avances, una branch es una línea de trabajo independiente y un pull request es la solicitud para integrar mejoras. Un archivo .pbix de Power BI es binario, por lo que Git no puede mostrar qué cambió por dentro. El formato .pbip soluciona esto guardando el proyecto en archivos de texto legibles. En el caso de Grafana, el versionado se logra exportando el cuadro de mando como un archivo JSON que Git puede comparar línea por línea.

Parte B · Práctica guiada: los mismos datos en dos herramientas

Paso 1 · Bronze: datos crudos



Se ejecutó el simulador de telemetría de un inversor solar de 5 kWp, generando registros de potencia, irradiancia y temperatura cada cinco minutos durante tres días. El archivo CSV original se conserva sin modificaciones para garantizar la trazabilidad de los datos.

Paso 2 · Silver: reglas de calidad

```
PS D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI> python ETL/limpiar_datos.py
Columnas encontradas: ['ts', 'dispositivo_id', 'p_ac_kw', 'irradiancia_wm2', 'temp_modulo_c']
Registros originales: 884

Proceso de limpieza terminado.
REPORTE DE CALIDAD DE DATOS - FASE SILVER

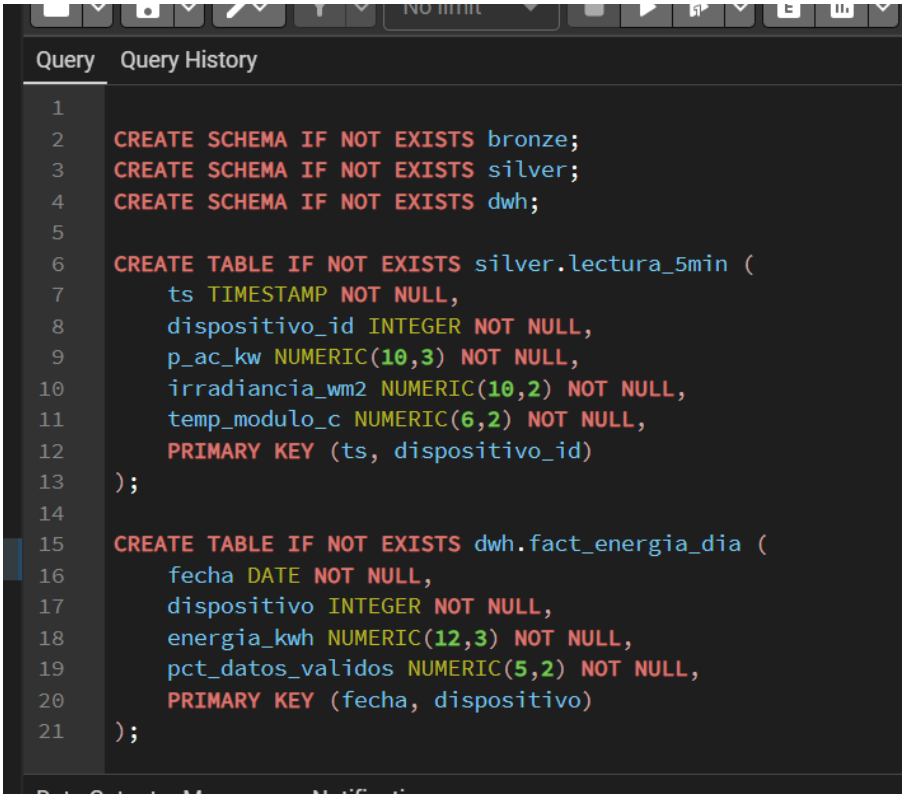
Archivo de entrada: D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI\Data\Bronze\telemetria.csv
Archivo de salida: D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI\Data\Silver\telemetria_limpia.csv

Registros originales: 884
Registros con campos vacíos o inválidos: 24
Registros fuera de rango: 16
Duplicados eliminados: 20
Registros válidos finales: 824
Registros rechazados totales: 60
Porcentaje de registros válidos: 93.21%

CSV limpio: D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI\Data\Silver\telemetria_limpia.csv
Reporte: D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI\Reports\calidad_datos.txt
PS D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI>
```

Se procesaron 884 registros mediante reglas de calidad para detectar datos faltantes o inválidos, valores fuera de rango y duplicados. El proceso generó un archivo limpio con 824 registros y un reporte que indica un porcentaje de datos válidos del 93,21 %.

Paso 3 · Gold: carga en PostgreSQL

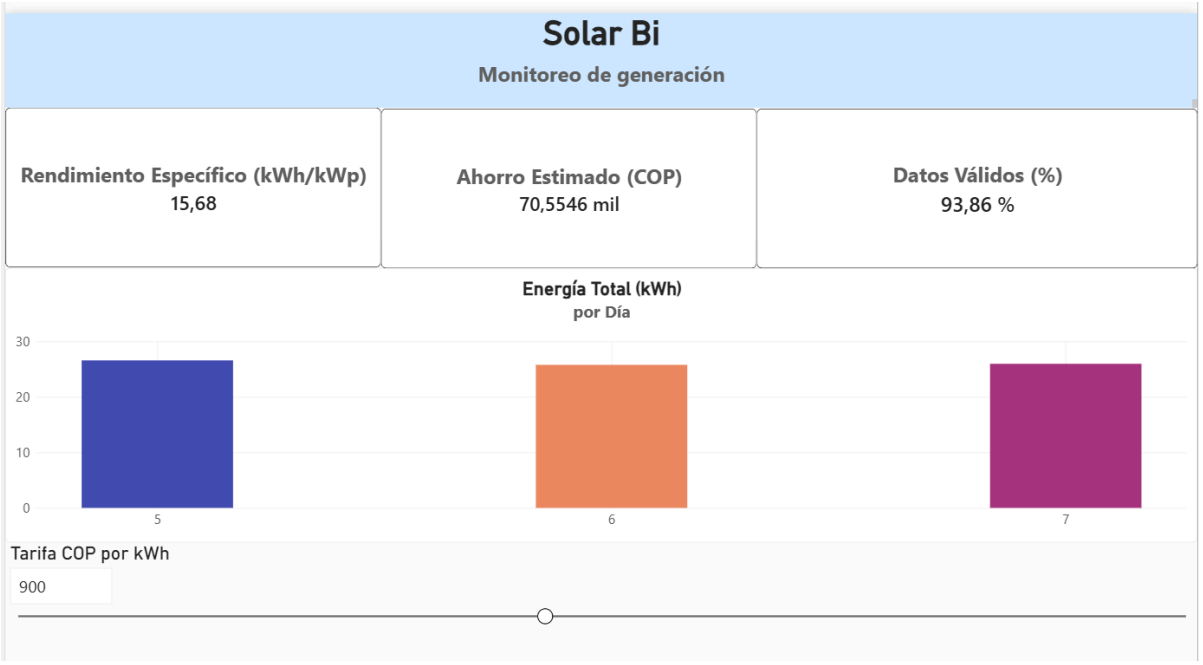


```
PS D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI> python ETL/run_etl.py
Iniciando proceso ETL...
Registros originales: 878
Registros limpios: 824
Grupos diarios calculados: 3
Usuario PostgreSQL [postgres]:
Contraseña de PostgreSQL:

ETL finalizado correctamente.
Lecturas válidas procesadas: 824
Resúmenes diarios procesados: 3
Filas almacenadas en Silver: 824
Filas almacenadas en Gold: 3
PS D:\users\Luz Estela\Desktop\Laurinchis\phyton\Practica BI>
```

Se implementó un proceso ETL en Python que carga las lecturas limpias en PostgreSQL y calcula un resumen diario de energía generada y porcentaje de datos válidos. El proceso se ejecutó dos veces y los conteos se mantuvieron en 824 registros en Silver y 3 registros en Gold, sin duplicar filas.

Paso 4 · Power BI

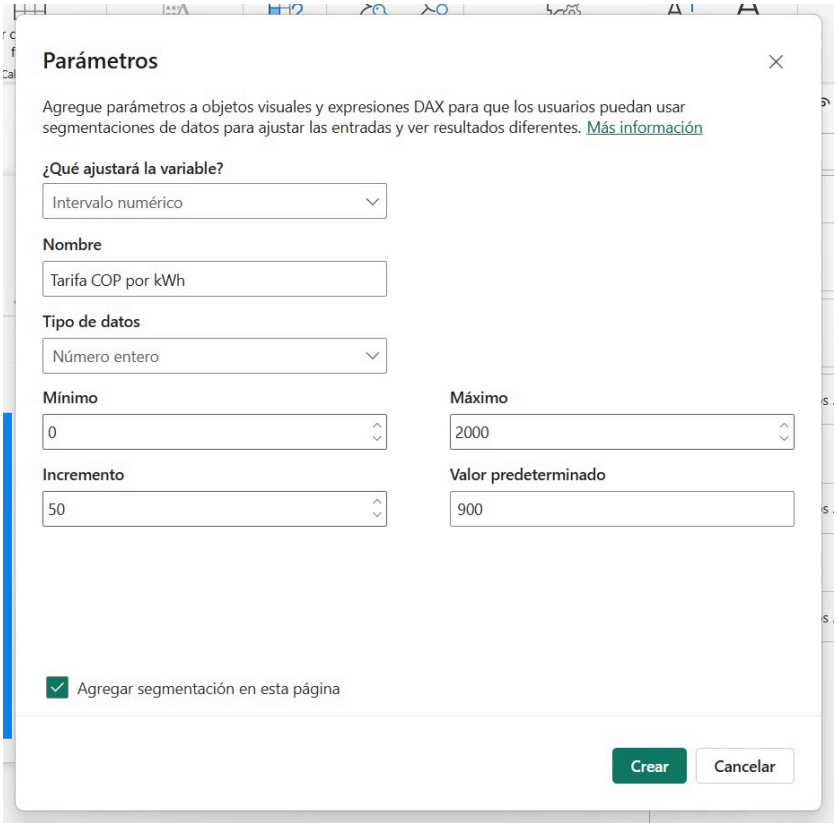


```
1  Energía Total (kWh) =
2  SUM('dwh fact_energia_dia'[energia_kwh])
```

fecha	dispositivo	energia_kwh	pct_datos_validos
lunes, 5 de octubre de 2026	1	26,605	95,52
martes, 6 de octubre de 2026	1	25,8	92,54
iércoles, 7 de octubre de 2026	1	25,989	93,52

```
1  Rendimiento Específico (kWh/kWp) =
2  DIVIDE(
3      [Energía Total (kWh)], 5, 0
4  )
```

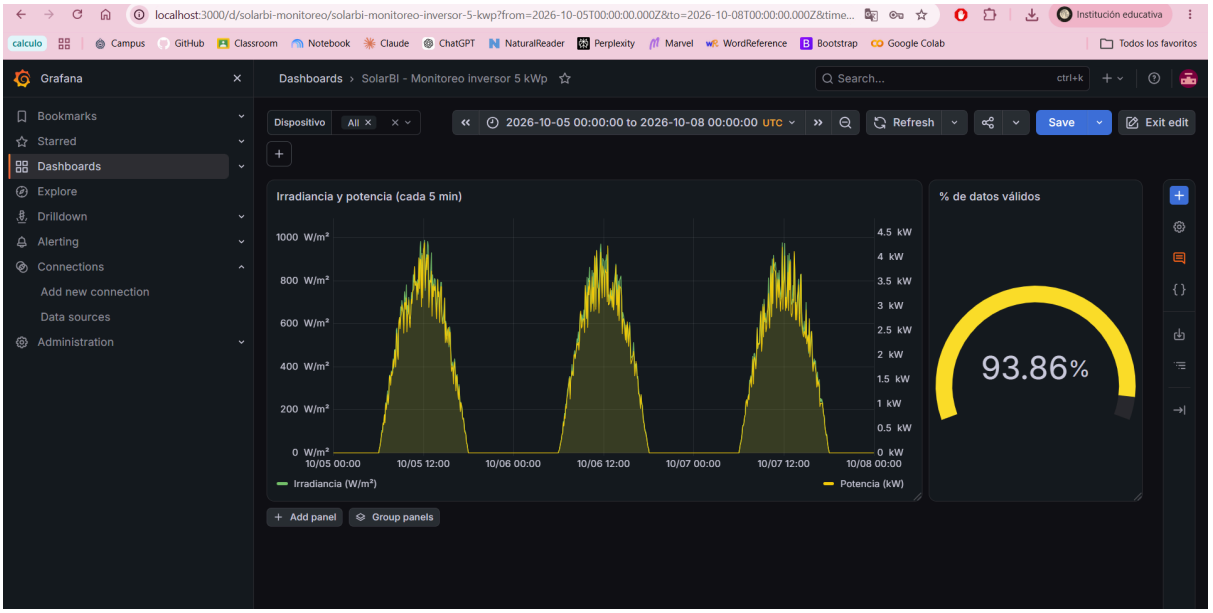
```
1  Datos Válidos (%) =
2  DIVIDE(
3      AVERAGE('dwh fact_energia_dia'[pct_datos_validos]), 100, 0
4  )
```

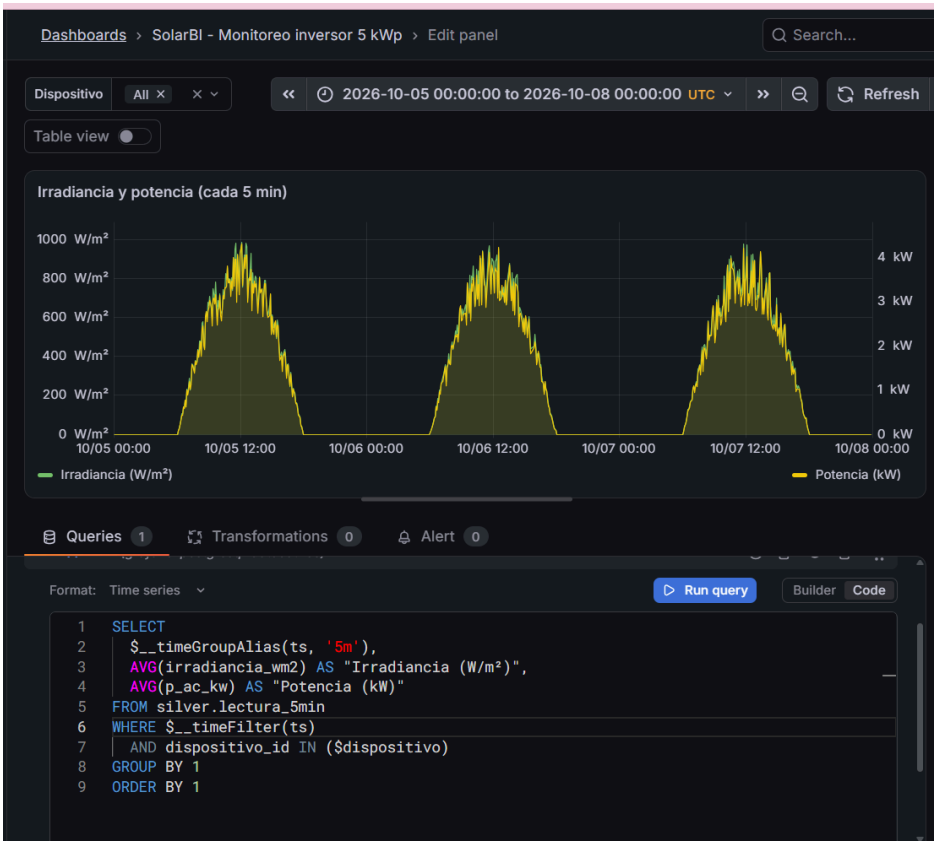


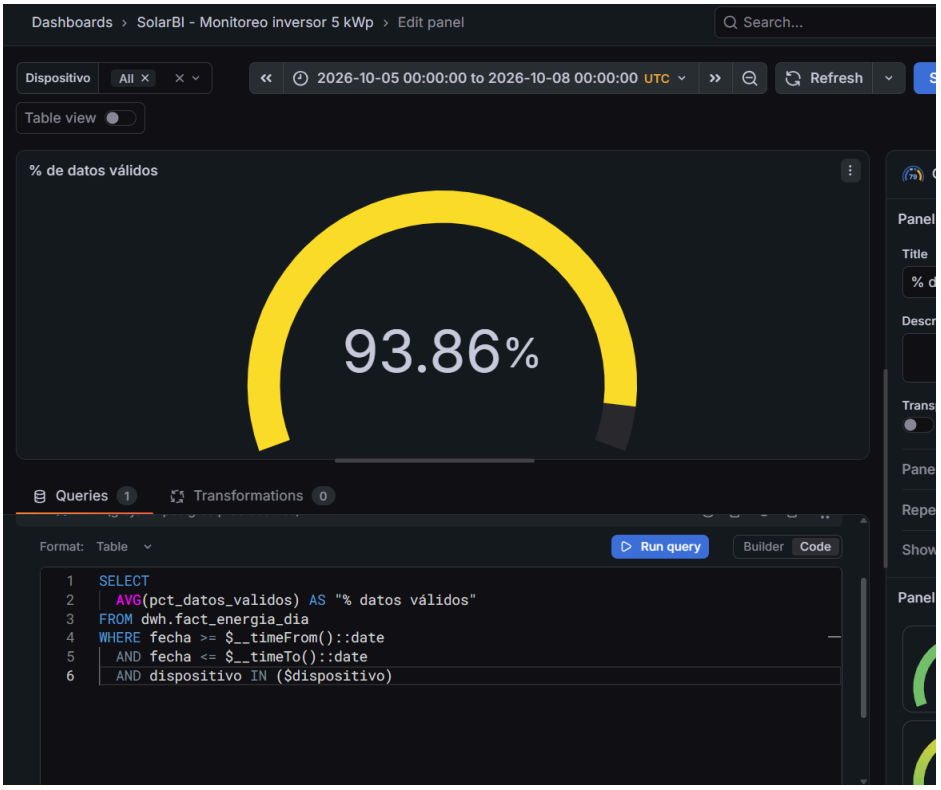
Power BI Desktop se conectó a PostgreSQL (base practica_bi) y leyó dwh.fact_energia_dia. La página muestra el rendimiento específico en kWh/kWp, un gráfico de columnas con la energía diaria y el ahorro estimado en COP, además del porcentaje de datos válidos. El ahorro se calcula con un parámetro de tarifa (Tarifa COP por kWh) que el usuario puede variar entre 900 y 2000 COP/kWh mediante un segmentador, y que en la captura está fijado en 900 COP/kWh. El rango toma como referencia las tarifas de energía publicadas por EPM

para septiembre de 2026, que van de unos 895 COP/kWh (residencial estrato 4) a unos 1,150 COP/kWh (comercial e industrial). Todas las cifras salen de medidas DAX.

Paso 5 · Grafana







Se instaló Grafana OSS en el equipo, para conectarse a grafana se hizo por medio de la dirección http://localhost:3000/ , se conectó a PostgreSQL y se construyó un tablero con una serie de tiempo de irradiancia y potencia cada 5 minutos (con `$__timeFilter`), un gauge del porcentaje de datos válidos con umbrales rojo, amarillo (90 %) y verde (95 %), y la variable `dispositivo` aplicada en ambas consultas.

El gauge marca 93.86 %, ligeramente distinto al 93.21 % del reporte de calidad porque promedia el porcentaje de cada día. El JSON del tablero está en `grafana/dashboard_solarbi.json` del repositorio.

Paso 6 · Reflexión

Al evaluar ambas herramientas sobre la misma base de telemetría, Grafana demostró una superioridad clara para la monitores de tiempo real: su auto-refresh constante, el filtrado directo en base de datos mediante variables ($dispositivo) y su sistema de umbrales alertan instantáneamente sobre fallas operativas o caídas de irradiancia en planta. Por su parte, Power BI resolvió de manera más robusta la analítica de negocio, permitiendo modelar relaciones, calcular variables complejas como el Yield y proyectar ahorros financieros en COP reaccionando a parámetros de tarifa.

Lo más complejo del proceso consistió en garantizar la congruencia entre ambas herramientas, evitando diferencias en los cálculos al procesar datos de 5 minutos en Grafana frente a la agregación diaria en Power BI. Sin duda, seleccionaremos Grafana para el operador o ingeniero de planta por su velocidad y enfoque operativo, y Power BI para el

directivo o tomador de decisiones, quien requiere una visión macro del rendimiento y retorno financiero del activo solar.

Referencias bibliográficas

[1] "Grafana OSS and Enterprise," Grafana Labs. Accessed: Oct. 09, 2026. [Online]. Available: https://grafana.com/docs/grafana/latest/

[2] "PEP 249 – Python database API specification v2.0," Python Enhancement Proposals (PEPs). Accessed: Oct. 09, 2026. [Online]. Available: https://peps.python.org/pep-0249/

[3] JulCsc, "Power BI documentation," Microsoft.com. Accessed: Oct. 09, 2026. [Online]. Available: https://learn.microsoft.com/power-bi/

[4] "5.10. Schemas," PostgreSQL Documentation. Accessed: Oct. 09, 2026. [Online]. Available: https://www.postgresql.org/docs/current/ddl-schemas.html

[5] "PostgreSQL data source," Grafana Labs. Accessed: Oct. 09, 2026. [Online]. Available: https://grafana.com/docs/grafana/latest/datasources/postgres/

[6] "Tarifas de energía EPM - 2026: consulta y compara," Portal EPM. Accessed: Oct. 09, 2026. [Online]. Available: https://www.epm.com.co/clientesyusuarios/energia/tarifas-energia/

[7] “INSERT,” PostgreSQL Documentation. Accessed: Oct. 09, 2026. [Online]. Available: https://www.postgresql.org/docs/current/sql-insert.html

[8] “Referencia de expresiones de análisis de datos (DAX),” Microsoft.com. Accessed: Oct. 09, 2026. [Online]. Available: https://learn.microsoft.com/es-es/dax/